Device control: Any computer system requires various devices. But to make the devices usable, there should be a device driver and this layer provides that functionality. There are various types of drivers present, like graphics drivers, a Bluetooth driver, audio/video drivers and so on.

Networking: Networking is one of the important aspects of any OS. It allows communication and data transfer between hosts. It collects, identifies and transmits network packets. Additionally, it also enables routing functionality.

Dynamically loadable kernel modules

We often install kernel updates and security patches to make sure our system is up-to-date. In case of MS Windows, a reboot is often required, but this is not always acceptable; for instance, the machine cannot be rebooted if is a production server. Wouldn't it be great if we could add or remove functionality to/from the kernel on-the-fly without a system reboot? The Linux kernel allows dynamic loading and unloading of kernel modules. Any piece of code that can be added to the kernel at runtime is called a 'kernel module'. Modules can be loaded or unloaded while the system is up and running without any interruption. A kernel module is an object code that can be dynamically linked to the running kernel using the 'insmod' command and can be unlinked using the 'rmmod' command.

A few useful utilities

GNU/Linux provides various user-space utilities that provide useful information about the kernel modules. Let us explore them.

lsmod: This command lists the currently loaded kernel modules. This is a very simple program which reads the */proc/modules* file and displays its contents in a formatted manner.

insmod: This is also a trivial program which inserts a module in the kernel. This command doesn't handle module dependencies.

rmmod: As the name suggests, this command is used to unload modules from the kernel. Unloading is done only if the current module is not in use. *rmmod* also supports the *-f* or *--force* option, which can unload modules forcibly. But this option is extremely dangerous. There is a safer way to remove modules. With the *-w* or *--wait* option, *rmmod* will isolate the module and wait until the module is no longer used.

modinfo: This command displays information about the module that was passed as a command-line argument. If the argument is not a filename, then it searches the */lib/modules/<version>* directory for modules. *modinfo* shows each attribute of the module in the field:value format.

Note: *<version>* is the kernel version. We can obtain it by executing the *uname -r* command.

dmesg: Any user-space program displays its output on the standard output stream, i.e., */dev/stdout* but the kernel uses a different methodology. The kernel appends its output to the ring buffer, and by using the 'dmesg' command, we can manage the contents of the ring buffer.

Preparing the system

Now it's time for action. Let's create a development environment. In this section, let's install all the required packages on an RPM-based GNU/Linux distro like CentOS and a Debian-based GNU/Linux distro like Ubuntu.

Installing CentOS

First install the *gcc* compiler by executing the following command as a root user:

```
[root]# yum -y install gcc
```

Then install the kernel development packages:

```
[root]# yum -y install kernel-devel
```

Finally, install the 'make' utility:

```
[root]# yum -y install make
```

Installing Ubuntu

First install the *gcc* compiler:

```
[mickey] sudo apt-get install gcc
```

After that, install kernel development packages:

```
[mickey] sudo apt-get install kernel-package
```

And, finally, install the 'make' utility:

```
[mickey] sudo apt-get install make
```

Our first kernel module

Our system is ready now. Let us write the first kernel module. Open your favourite text editor and save the file as *hello.c* with the following contents:

```
#include <linux/kernel.h>
#include <linux/module.h>

int init_module(void)
{
     printk(KERN_INFO "Hello, World !!!\n");

     return 0;
}

void cleanup_module(void)
{
     printk(KERN_INFO "Exiting ...\n");
}
```